

WEEK 3: PYTHON-BASED MACHINE LEARNING MODEL DEVELOPMENT AND EVALUATION PLAN

AI-Based Pothole Detection and Road Condition Analysis Using Python

Student Name	Rajvardhan Somase
College	Ajeenkya DY Patil School of Engineering
Program	B.Tech – Artificial Intelligence & Data Science
Task	Week 3: Python-Based Machine Learning Model Development and Evaluation Plan

1. Problem Definition and Motivation

Road potholes may damage vehicles and create road-safety risks. The proposed Python-based computer vision system will detect potholes in road images and locate them using bounding boxes. This is an object-detection task. Week 3 focuses on designing a development and evaluation plan; no model training or measured performance is claimed.

2. Project Objectives and Scope

- Define the input road images, pothole annotations and expected detection outputs.
- Plan data cleaning, preprocessing, dataset splitting and model training.
- Select an appropriate model and outline hyperparameter tuning.
- Define validation methods, performance metrics and error analysis.
- Describe possible deployment and model maintenance.

3. Dataset and Preprocessing Plan

Use appropriately licensed images covering different roads, lighting conditions, weather and pothole sizes. Inspect image quality, remove duplicates and corrupted images, and review bounding-box labels. Split images by source or road sequence to prevent leakage. Apply resizing and normalization appropriate for the selected model; apply augmentation only to training data.

Stage	Proposed Approach
Annotation	Label potholes with consistent bounding boxes.
Dataset Split	Illustrative 70% training, 15% validation and 15% held-out testing.
Data Quality	Check annotation errors, missing labels and image diversity.
Preprocessing	Resize, normalize and augment training images as appropriate.

4. Model Selection and Justification

Ultralytics YOLO is proposed as the primary object-detection model because it predicts pothole locations and confidence scores. A smaller pretrained model can be tested first on limited hardware, with larger variants considered if resources allow. A simple classification model could provide a baseline for pothole presence but cannot locate individual potholes.

5. Training and Hyperparameter Tuning Strategy

- Initialize the selected YOLO model with suitable pretrained weights.
- Train using the training split and monitor results on the validation split.
- Compare learning rate, batch size, image size, epochs and augmentation settings.
- Record model configuration, random seed, software and hardware.
- Select the checkpoint using validation performance and evaluate once on the held-out test set.

6. Evaluation Metrics and Validation

Use validation results for model selection and an untouched test set for final reporting. Group related road images to reduce data leakage. Report confidence and IoU thresholds when comparing results.

Metric	Purpose
--------	---------

Precision	Measures how many detected potholes are correct.
Recall	Measures how many actual potholes are detected.
F1-score	Balances precision and recall at a selected threshold.
mAP@0.5 / mAP@0.5:0.95	Evaluates object-detection quality across IoU thresholds.
Inference Time	Measures image processing speed on specified hardware.
Error Analysis	Reviews missed potholes, false alarms and inaccurate bounding boxes.

Accuracy and ROC AUC can be considered for an optional image-level classification baseline; mAP and IoU-based measures are more appropriate for the main detection task.

7. Planned Machine Learning Workflow

Problem Definition → Data Collection and Annotation → Data Cleaning → Train/Validation/Test Split → Preprocessing and Augmentation → YOLO Training → Hyperparameter Tuning → Test Evaluation → Visualization → Future Deployment

8. Challenges and Mitigation

Challenge	Mitigation
Low light or blurry images	Use diverse images and careful augmentation.
Small or partially hidden potholes	Review annotations and image resolution.
Shadows causing false positives	Include representative negative examples and review errors.
Overfitting or data leakage	Use grouped splits, validation and regularization.
Limited computing resources	Start with smaller pretrained models.

9. Deployment and Maintenance Concept

A future Streamlit or FastAPI application could accept road images, display bounding boxes and export reports. Benchmark inference speed on target hardware, validate on unseen roads, monitor errors, version datasets and models, and retrain only after validating newly labeled images.

10. Proposed 35-Hour Timeline

Activity	Planned Work	Hours
Problem Definition	Define task, scope and data requirements	4
Preprocessing Plan	Data cleaning, labeling and split strategy	5
Model Selection	Compare candidate models and design experiments	6
Training Strategy	Training and hyperparameter tuning plan	6
Evaluation Plan	Validation, metrics and error analysis	5
Workflow & Deployment	Diagram and deployment concept	4
Documentation	Report writing and review	5
TOTAL		35

11. Expected Outcomes

- A clear plan for building a Python-based pothole detector.
- A documented dataset and preprocessing strategy.
- A justified model choice and reproducible training plan.
- Appropriate validation, evaluation and error-analysis methods.
- A foundation for future visualization and deployment.

12. Conclusion

This Week 3 report provides a structured plan for model development and evaluation. It defines the data pipeline, YOLO model strategy, training and tuning approach, evaluation metrics, workflow and possible deployment. Actual implementation and performance measurements are reserved for future project stages.

13. References / Planned Resources

- Ultralytics YOLO Documentation — <https://docs.ultralytics.com/>

- PyTorch Documentation — <https://pytorch.org/docs/stable/>
- OpenCV Documentation — <https://docs.opencv.org/>
- scikit-learn Model Evaluation — https://scikit-learn.org/stable/modules/model_evaluation.html